

claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_f9255361-69d\agent-  
a63be622745e9d151.jsonl`  
- SHA-256: `d0b9d3c59954dab2d22c56413b2215979987a529d359b5a1acf0d4796819754e`  
- Source modified: `2026-06-22T09:35:44+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_f9255361-69d`  
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`
```

Transcript

1. user (2026-06-22T09:32:46.032Z)

You are auditing ONE doc in the KhelSutra brand/site repo for staleness, as part of a "are all the docs up to date?" sweep.

Repo root: C:\Users\avidu\Projects\kheelsutra (bash path: /c/Users/avidu/Projects/kheelsutra)
The repo is on `master`, freshly synced to remote HEAD ? the latest merged PR is #94. Today is 2026-06-22.

DOC TO AUDIT: docs/REQUEST_ACCESS_FORM.md

Your job: read the doc IN FULL, extract every concrete, falsifiable claim, and VERIFY each against the LIVE repo ? do NOT trust the doc's own wording. Hard discipline rule: a claim is "correct" ONLY when you have file:line or command-output evidence for it; if you cannot verify it, mark verdict "unverifiable" ? never guess, never assume the doc is right. Re-check against actual code/git.

Kinds of claims to check:

- File / path / route references ("served at /capture", "see src/store.js", "public/og.png", "systemd kheelsutra-site") -> does the path or route ACTUALLY exist now? Confirm with git ls-files / Read / the routing in site/server.js + site/src/index.js.
- PR numbers and counts ("PRs #1-#8 merged", "20 tests green", "6 languages", "3-step wizard", "P0..P5 done") -> verify against `git -C <root> log`, the actual test output/count, and the code.
- Status markers (DONE / pending / IN PROGRESS / DRAFT / "?" / "owner-gated" / "NEXT") -> still accurate vs the code & git history? Is something marked pending that is actually shipped, or vice-versa?
- Dates, "live at <url>", "merged in #N".
- Whether the doc describes something now SUPERSEDED or fully shipped -> is it an archive candidate (a tracker whose work is complete, or content duplicated by docs/archives/)? Note: docs/archives/FRIENDLY_AND_MULTILINGUAL.md already exists as the archived multilingual tracker.

Tools: Read the doc + every cited file; Grep/Glob to locate code; Bash for `git -C /c/Users/avidu/Projects/kheelsutra log --oneline`, `... ls-files`, and greps. To check a test count, you may run the suite (`cd /c/Users/avidu/Projects/kheelsutra/site && node --test` or `npm test`), but prefer cheap checks. Verify against master as it stands NOW.

Severity: high = a factually wrong claim a reader would act on (wrong path/route, a "DONE" that isn't, a wrong count/PR that misleads). medium = outdated-but-harmless (stale PR range, old "where we are", superseded framing). low = cosmetic.

Do NOT modify any file ? audit only. Return ONLY the structured object (doc, overallStatus, findings[], summary).

2. assistant (2026-06-22T09:32:48.913Z)

I'll audit the doc. Let me start by reading it in full and syncing the repo.

3. user (2026-06-22T09:32:50.357Z)

```
1      # Request-Access form ? segmented lead capture + setup discovery
2
3      > **Status:** `IN PROGRESS` . **Owner:** Avi . **Created:** 2026-06-20 . **Last
updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
5      > **Relation:** first tracked work item in the `khelsutra` repo; mirrors the engine
repo's
6      > `PROJECT_DOC_TEMPLATE` discipline (status header . decisions-locked . $6 tracker .
DoD).
7      > Feeds the platform direction in the engine repo's `PLATFORM_ARCHITECTURE.md $3`
(StorageBackend /
8      > ComputeBackend) and `VIDEO_LOCALITY_MODEL.md` ? see $7.
9
10     ---
11
12     ## 0. TL;DR
13     Replace the landing page's `mailto:` "Request access" CTA with a real form that (a)
segments
14     **academies vs individual enthusiasts**, (b) asks the two questions that actually decide
how we'd
15     serve a user ? ***"do you have a GPU machine?"*** (compute placement) and ***"where do your
videos
16     live?"*** (storage backend) ? and (c) keeps a free-text message. Submissions are stored
in our own
17     **Cloudflare D1** via a small **Worker** (`POST /api/request-access`). The form doubles
as PMF +
18     architecture research.
19
20     ## 1. Problem & goal
21     Today the CTA is `mailto:hello@khelsutra.guru` ? zero structure, nothing collected, no
segmentation.
22     We want **structured, queryable** interest signals from two distinct audiences, captured
the moment
23     someone raises a hand, **without shipping lead data to a third party** (privacy-first
ethos).
24     **Goal:** a clean on-brand form whose responses land in our own Cloudflare account,
queryable later.
25
26     ## 2. Decisions locked
27     - **Backend = Cloudflare Worker + D1** (owner decision, 2026-06-20). Data stays in our
Cloudflare
28     account; queryable; no third party. (Alternatives considered: hosted form
(Tally/Formspre
29     third-party data), Worker?email-only (no DB), structured `mailto` (no collection).
Rejected for
30     the reasons in parens.)
31     - **Two audience segments:** `academy` and `individual` ? the form lightly adapts
(academy also
32     asks club name + #courts).
33     - **The two discovery questions are first-class** (not optional polish): GPU/compute +
storage ?
34     they map 1:1 onto the BY0-compute / BY0-storage architecture forks ($7).
35     - **Free-text message retained** ("Anything else?").
36     - **Privacy:** store only what's submitted + a coarse country (from
`request.cf.country`); **no raw
37     IP**; consent checkbox for "email me about the alpha"; this data is
**operations/contact only**,
38     never a training source (same guardrail as the engine repo).
39     - **Deploy model changes:** the site moves from dashboard direct-upload ? **Wrangler**
(Worker with
40     static assets + D1 binding). Documented in $5; owner runs the `wrangler` deploy (our
```

```

account/auth).
41
42    ## 3. Form spec
43    | Field | Type | Notes |
44    |---|---|---|
45    | Segment | radio | `academy` \ | `individual` (required) |
46    | Name | text | required |
47    | Email | email | required (so we can reply) |
48    | City | text | optional |
49    | Club / academy name | text | shown for `academy` |
50    | Courts recorded | select | `1` \ | `2-4` \ | `5+` (academy; optional) |
51    | **GPU machine for local processing?** | radio | `yes` \ | `unsure` \ | `no` \ | `prefer-
cloud` |
52    | **Where do your recordings live / how would you share them?** | checkboxes (multi) |
`gdrive` \ | `onedrive` \ | `dropbox` \ | `bucket` (S3/GCS) \ | `local` (SD/drive) \ | `unsure` |
53    | Message | textarea | free text ("Anything else?") |
54    | Consent | checkbox | "Okay to email me about the KhelSutra alpha" |
55    | `hp` | hidden honeypot | anti-spam; if filled ? silently drop |
56
57    ## 4. Design / architecture
58    - **Frontend** (`site/index.html`): the header + hero "Request access" buttons scroll to
the
59    `#join` form; `#join` becomes the form. On submit, JS `POST`s JSON to `/api/request-
access`.
60    **Progressive enhancement:** if the endpoint isn't live yet (pre-Worker), it falls
back to a
61    prefilled `mailto:hello@` so the form is useful from the first re-upload.
62    - **Worker** (`/api/request-access`): validates (email present, honeypot empty), inserts
into D1,
63    returns `200 {ok:true}`; serves the static assets for all other routes.
64    - **D1 schema** (`access_requests`): `id, created_at, segment, name, email, city,
org_name, courts,
65    has_gpu, storage (csv), message, consent, country, user_agent`.
66    - **Anti-spam:** honeypot first; add **Cloudflare Turnstile** in P2 if spam appears.
67    - **Notify (P2):** optional email ping to `hello@` per submission (MailChannels /
Cloudflare Email).
68
69    ## 5. Deploy (owner-run; our Cloudflare account)
70    ```
71    wrangler d1 create khelsutra-access          # once; copy the database_id into
wrangler.toml
72    wrangler d1 execute khelsutra-access --file=site/schema.sql # create the table
73    wrangler deploy                                # ships the Worker + static assets + D1
binding
74    ```
75    > Until the Worker ships, the form works via the `mailto` fallback (re-upload `site/` as
today).
76    > Git?Cloudflare auto-deploy still intentionally not connected.
77
78    ## 6. Deliverables & progress tracker ? source of truth
79    Legend: ? Todo . ? In progress . ? Done. One PR per row off `origin/master`.
80    | ID | Deliverable | Gated? | Status | PR |
81    |---|---|---|---|---|
82    | P0 | This design doc + `NEXT_STEPS.md` | No | ? | #2 |
83    | P1 | Form UI (segments + GPU + storage + free text + consent) w/ `mailto` fallback |
No | ? | #2 |
84    | P2 | Worker + D1 + `wrangler.toml` + `schema.sql` + **unit tests (10 green)** | ? | ?
| **LIVE on Cloudflare** (Worker `hidden-king-khelsutra` + D1 `khelsutra-access`; persists +
emails per submission) |
85    | P3 | Anti-spam (Turnstile) + per-submission email notify | No | ? | **email-notify
LIVE** (per-submission to `hello@`); Turnstile still ? |
86    | P4 | Tiny admin/export view (auth'd) to read responses | owner | ? | ? |
87
88    ## 7. Why this matters beyond lead capture
89    The two questions are **the architecture forks** from the engine repo's BYO analysis:

```

```

90 - **GPU?** ? compute placement: desktop/local (their GPU) vs hosted vs zero-infra-BYO
91   (`PLATFORM_ARCHITECTURE.md §3.2`, `DESKTOP_APP_PLAN.md` P0 spike).
92 - **Storage?** ? which `StorageBackend` + whether autosync is worth building
93   (`PLATFORM_ARCHITECTURE.md §3.1`, the cloud-storage/autosync thread).
94 So early responses directly inform what to build next ? not just who to email.
95
96 ## 8. Open questions
97 1. Turnstile now or wait for spam? (Default: honeypot now, Turnstile if needed.)
98 2. Email-notify per submission, or just query D1 weekly? (Default: notify in P2.)
99 3. Do we localize the form (Kannada/Hindi) for academies now or at beta? (Default:
English now.)
100
101 ## 9. Definition of done
102 - [ ] Form replaces the `mailto` CTA, segments academy/individual, asks GPU + storage +
free text.
103 - [ ] Submissions persist to Cloudflare D1 via the Worker; honeypot drops bots.
104 - [ ] Privacy posture honored (no raw IP; consent; contact-only).
105 - [ ] Deployed live on khelsutra.guru; a test submission appears in D1.
106 - [ ] Doc tracker + DoD updated; archived when P1?P3 land.
107

```

4. user (2026-06-22T09:32:50.873Z)

```

50b2bd7 chore(repo): ignore local runtime artifacts + correct stale NEXT_STEPS claims (#94)
e2570e4 docs: archive the Friendly & Multilingual tracker (engineering complete) (#93)
0bef61e docs(friendly-multilingual): close out the chapter ? log #89/#90/#91 + strengthen fitNav
tests (#92)
9f77ddc test(site): browser layout-regression CI guard + auto-collapse the nav to fit every
locale (#91)
4415fe8 feat(site): localize remaining home sections (6 langs) + smooth the responsive header
(#90)
26c9da1 feat(site): localize the hero demo mockup + #how feature block (6 languages) (#89)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
(#88)
cf59fbf feat(site): add es/da/id to the marketing site (parity with the SPA's 6 languages) (#87)
17242a9 feat(site): localize own-model body content to kn/hi (#86)
7b04f02 deploy(kit): align with the live-verified droplet deploy (+ the gemini-default fix)
(#85)
5e5a2ac docs(friendly-multilingual): close out the chapter (§9 DoD + UI/UX + B7 deferred) (#84)
4c0f35c fix(site): UI/UX ? compact language switcher + header overflow fix + localized hero
(#83)
0fae364 feat(web+site): A2 ? localized WhatsApp share for the reel (SPA + site) (#82)
1e52cfd deploy(droplet): tailored alpha kit for the CPU droplet (engine api. side) (#81)
64076a5 test(web): add a coverage floor (CI-gated) + i18n regression tests for the #77 audit
(#80)

```

5. assistant (2026-06-22T09:32:53.968Z)

This doc is the "Request-Access form" tracker. Let me verify its concrete claims against the live repo. Let me look at the site structure, the Worker, schema, tests, and the relevant PRs.

6. user (2026-06-22T09:32:56.229Z)

```

site/README.md
site/make_og.py
site/package-lock.json
site/package.json
site/public/capture.html
site/public/flyer-hi.html
site/public/flyer-kn.html
site/public/flyer.html
site/public/i18n.js
site/public/img/annotate-enable.png
site/public/img/annotate-tool.png

```

```
site/public/img/flyer-qr.svg
site/public/img/studio-en.png
site/public/img/studio-hi.png
site/public/img/studio-kn.png
site/public/index.html
site/public/locales/da/common.json
site/public/locales/en/common.json
site/public/locales/es/common.json
site/public/locales/hi/common.json
site/public/locales/id/common.json
site/public/locales/kn/common.json
site/public/og.png
site/public/own-model.html
site/public/robots.txt
site/public/video/tool-cuj.mp4
site/schema.sql
site/scripts/copy-locales.mjs
site/scripts/droplet-smoke.sh
site/scripts/parity-check.sh
site/scripts/provision-vm.sh
site/scripts/teardown-vm.sh
site/server.js
site/src/assets.test.js
site/src/deploy-config.test.js
site/src/i18n.shim.test.js
site/src/index.js
site/src/index.test.js
site/src/layout.browser.mjs
site/src/locales-drift.test.js
site/src/notify.js
site/src/server.test.js
site/src/store.js
site/src/ux.dom.test.js
site/wrangler.toml
---TOPLEVEL---
docs/REQUEST_ACCESS_FORM.md
site/schema.sql
site/wrangler.toml
```

7. user (2026-06-22T09:32:56.974Z)

```
aa8d5e9 feat(site): A6 ? plain-language Request-Access GPU/storage questions (#75)
4627830 docs(deploy): alpha deploy kit ? make Studio a shareable page (split: Pages + api.*
tunnel + CF Access) (#65)
8140ffd feat(site): show an obvious 'Sending?' state on request-access submit (#36)
d537d19 feat(web): per-rally MVP P3 ? deterministic parseQuery() query bar (traced +
adversarially hardened) (#28)
0ce6a30 feat(site): random start point for the sample reel + form-consistent "How do I join"
(#20)
a1c0d89 fix(site): match wrangler name to the git-connected Worker (hidden-king-khelsutra) (#5)
a45e4af chore(site): wire D1 database_id (khelsutra-access) for the deploy binding (#4)
f699847 feat(site): Request-Access Worker + D1 backend (+ 10 unit tests) (#3)
2274faa feat(site): segmented Request-Access form (academy/individual + GPU + storage discovery)
(#2)
```

8. assistant (2026-06-22T09:33:00.883Z)

Note: the doc references `site/index.html`, `site/schema.sql`, `site/wrangler.toml`. The actual frontend appears to be at `site/public/index.html`. Let me verify the key files: the Worker code, schema, wrangler.toml, the form HTML, and the tests.

9. user (2026-06-22T09:33:01.508Z)

```
1      -- D1 schema for Request-Access submissions.
```

```

2      -- Apply: npx wrangler d1 execute khelsutra-access --remote --file=schema.sql
3      CREATE TABLE IF NOT EXISTS access_requests (
4          id          INTEGER PRIMARY KEY AUTOINCREMENT,
5          created_at  TEXT NOT NULL DEFAULT (datetime('now')),
6          segment     TEXT,          -- 'academy' | 'individual'
7          name        TEXT,
8          email       TEXT NOT NULL,
9          city        TEXT,
10         org_name     TEXT,          -- academy / club name
11         courts       TEXT,          -- '1' | '2-4' | '5+'
12         has_gpu      TEXT,          -- 'yes' | 'unsure' | 'no' | 'prefer-cloud'
13         storage      TEXT,          -- csv of gdrive,onedrive,dropbox,bucket,local,unsure
14         message      TEXT,          -- free text
15         consent      INTEGER DEFAULT 0,
16         country      TEXT,          -- coarse, from cf-ipcountry (no raw IP stored)
17         user_agent   TEXT
18     );
19

```

10. user (2026-06-22T09:33:01.991Z)

```

1      # Cloudflare Worker for khelsutra.guru ? static site + Request-Access form API.
2      #
3      # One-time setup (owner, our Cloudflare account):
4      #   npx wrangler d1 create khelsutra-access          # copy the printed database_id
below
5      #   npx wrangler d1 execute khelsutra-access --remote --file=schema.sql
6      #   npx wrangler deploy                             # or connect this repo via Workers
Builds (git)
7      #
8      name = "hidden-king-khelsutra" # the git-connected Worker that owns khelsutra.guru ?
keep in sync with Cloudflare
9      main = "src/index.js"
10     compatibility_date = "2025-06-01"
11
12     [assets]
13     directory = "./public"
14     binding = "ASSETS"
15
16     [[d1_databases]]
17     binding = "DB"
18     database_name = "khelsutra-access"
19     database_id = "cd02c6bf-4bd6-4a33-a333-ced86e36b511" # from `wrangler d1 create
khelsutra-access`
20
21     # Email-on-registration: Email Routing send binding -> a VERIFIED destination address.
22     [[send_email]]
23     name = "NOTIFY"
24     destination_address = "avi.dullu@gmail.com"
25
26     [vars]
27     NOTIFY_FROM = "noreply@khelsutra.guru"
28     NOTIFY_TO = "avi.dullu@gmail.com"
29

```

11. user (2026-06-22T09:33:02.383Z)

```

1      // Storage adapters for Request-Access submissions.
2      //
3      // PORTABILITY (see docs/PORTABILITY.md): the business logic (validate / toRow /
4      // handleRequest in index.js) is runtime-agnostic ? ALL database coupling lives here.
5      // The Cloudflare Worker uses d1Store(env.DB); an always-on VM uses sqliteStore(db)
6      // over node:sqlite (or better-sqlite3). Same schema.sql, same INSERT ? zero dialect
7      // change. To move hosts, swap the adapter, not the logic.
8

```

```

9      export const INSERT_SQL =
10      `INSERT INTO access_requests
11      (segment, name, email, city, org_name, courts, has_gpu, storage, message, consent,
country, user_agent)
12      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`;
13
14      export const insertArgs = (r) => [
15      r.segment, r.name, r.email, r.city, r.org_name, r.courts,
16      r.has_gpu, r.storage, r.message, r.consent, r.country, r.user_agent,
17      ];
18
19      // Cloudflare D1 (async): prepare(sql).bind(...args).run()
20      export function d1Store(DB) {
21      return {
22      async insertAccessRequest(r) {
23      await DB.prepare(INSERT_SQL).bind(...insertArgs(r)).run();
24      },
25      };
26      }
27
28      // node:sqlite / better-sqlite3 (sync): prepare(sql).run(...args). For the VM host.
29      export function sqliteStore(db) {
30      const stmt = db.prepare(INSERT_SQL);
31      return {
32      async insertAccessRequest(r) {
33      stmt.run(...insertArgs(r));
34      },
35      };
36      }
37

```

12. assistant (2026-06-22T09:33:06.290Z)

The schema matches. Now let me check the Worker route handler (index.js), the form HTML, and the tests count.

13. user (2026-06-22T09:33:07.246Z)

```

1      // Cloudflare Worker for khelsutra.guru ? serves the static site and handles the
2      // Request-Access form: POST /api/request-access -> validate -> insert into D1 (binding
DB).
3      // Pure helpers are exported so they can be unit-tested without a live Worker/D1.
4
5      import { d1Store } from "./store.js";
6      import { cfEmailNotify } from "./notify.js";
7
8      const SEGMENTS = ["academy", "individual"];
9      const GPU = ["yes", "unsure", "no", "prefer-cloud"];
10     const COURTS = ["1", "2-4", "5+"];
11     const STORAGE = ["gdrive", "onedrive", "dropbox", "bucket", "local", "unsure"];
12
13     function str(v, max) {
14     if (v == null) return null;
15     const s = String(v).trim();
16     return s ? s.slice(0, max) : null;
17     }
18
19     // Honeypot: humans never fill the hidden "company" field.
20     export function isBot(p) {
21     return !(p && typeof p.company === "string" && p.company.trim() !== "");
22     }
23
24     export function validate(p) {
25     if (!p || typeof p !== "object") return { ok: false, error: "bad payload" };
26     const email = String(p.email || "").trim();

```

```

27     if (!/^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$/ .test(email)) return { ok: false, error: "valid email
required" };
28     return { ok: true };
29 }
30
31 export function toRow(p, meta = {}) {
32     const storage = Array.isArray(p.storage)
33         ? p.storage.filter((s) => STORAGE.includes(s)).join(",")
34         : "";
35     return {
36         segment: SEGMENTS.includes(p.segment) ? p.segment : null,
37         name: str(p.name, 200),
38         email: str(p.email, 320),
39         city: str(p.city, 200),
40         org_name: str(p.org_name, 200),
41         courts: COURTS.includes(p.courts) ? p.courts : null,
42         has_gpu: GPU.includes(p.has_gpu) ? p.has_gpu : null,
43         storage,
44         message: str(p.message, 4000),
45         consent: p.consent ? 1 : 0,
46         country: str(meta.country, 8),
47         user_agent: str(meta.userAgent, 400),
48     };
49 }
50
51 function json(obj, status = 200) {
52     return new Response(JSON.stringify(obj), { status, headers: { "content-type":
"application/json" } });
53 }
54
55 export async function handleRequest(request, store, notify) {
56     let body;
57     try {
58         body = await request.json();
59     } catch {
60         return json({ ok: false, error: "invalid json" }, 400);
61     }
62     if (isBot(body)) return json({ ok: true }); // silently drop bots
63     const v = validate(body);
64     if (!v.ok) return json({ ok: false, error: v.error }, 400);
65
66     const r = toRow(body, {
67         country: request.headers.get("cf-ipcountry"),
68         userAgent: request.headers.get("user-agent"),
69     });
70
71     await store.insertAccessRequest(r);
72     try { await notify?.notify(r); } catch { /* email is best-effort; never fail the
request */ }
73
74     return json({ ok: true });
75 }
76
77 export default {
78     async fetch(request, env) {
79         const url = new URL(request.url);
80         if (url.pathname === "/api/request-access") {
81             if (request.method !== "POST") return json({ ok: false, error: "method not
allowed" }, 405);
82             return handleRequest(request, d1Store(env.DB), cfEmailNotify(env.NOTIFY,
env.NOTIFY_FROM, env.NOTIFY_TO));
83         }
84         return env.ASSETS.fetch(request); // everything else: static assets
85     },
86 };

```


14. user (2026-06-22T09:33:09.238Z)

```

exit=2
---grep request-access route---
site/src/assets.test.js:25:  assert.match(html, /\api\/request-access/, "form no longer posts
to /api/request-access");
site/src/assets.test.js:143:test("the 'How do I join?' FAQ points at the request-access form,
not email", () => {
site/src/assets.test.js:147:  assert.match(m[1], /request access/i, "the join FAQ should send
people to the request-access form");
site/src/assets.test.js:176:  // the sample REEL stays gated behind request-access ? NOT
embedded/playable here, no footage shown
site/src/assets.test.js:177:  assert.doesNotMatch(om, /<iframe/i, "the reel must NOT be embedded
here ? it's gated behind request-access");
site/src/assets.test.js:178:  assert.doesNotMatch(om, /youtube/i, "no YouTube embed on own-
model; the reel is shown only after request-access");
site/src/assets.test.js:431:  assert.match(enCatalog.site.faq.q7.a, /<a href="#join">request
access<\a>/, "en faq.q7.a lost the request-access link");
site/src/index.js:2:// Request-Access form: POST /api/request-access -> validate -> insert into
D1 (binding DB).
site/src/index.js:80:  if (url.pathname === "/api/request-access") {
site/src/index.test.js:25:  return new Request("https://khelsutra.guru/api/request-access", {
site/src/index.test.js:91:  const bad = new Request("https://khelsutra.guru/api/request-access",
{
site/src/index.test.js:138:  const res = await worker.fetch(new
Request("https://khelsutra.guru/api/request-access"), { DB: fakeDB() });
site/src/server.test.js:82:test("[VM host] POST /api/request-access persists to SQLite", async
() => {
site/src/server.test.js:83:  const res = await fetch(`${base}/api/request-access`, {
site/src/server.test.js:101:  const bot = await fetch(`${base}/api/request-access`, {
site/src/server.test.js:106:  const bad = await fetch(`${base}/api/request-access`, {
site/src/server.test.js:117:test("[VM host] GET /api/request-access -> 405", async () => {
site/src/server.test.js:118:  assert.equal((await fetch(`${base}/api/request-access`)).status,
405);
site/src/ux.dom.test.js:16:// Boot the real page; `fetchImpl` decides how /api/request-access
resolves. window.setInterval is
site/public/index.html:52:  /* in-flight spinner for the request-access submit */
site/public/index.html:738:      var r = await fetch('/api/request-access', {method:'POST',
headers:{'content-type':'application/json'}, body: JSON.stringify(p)});

```

15. assistant (2026-06-22T09:33:15.100Z)

Several important discrepancies emerging. Let me note:

1. The doc says the route is `POST /api/request-access` ? CONFIRMED at index.js:80.
2. The doc says frontend is `site/index.html` ? but the actual file is `site/public/index.html` (site/index.html does NOT exist, exit=2).
3. The doc says the honeypot field is `hp` (\$3) ? but the actual honeypot field is `company` (index.js:21 `isBot` checks `p.company`).

Let me verify the honeypot field name in the HTML, check the actual test count, and verify the `mailto` fallback claim.

16. user (2026-06-22T09:33:17.277Z)

```

602:      <input class="hp" type="text" name="company" tabindex="-1" autocomplete="off" aria-
hidden="true">
620:      <div><a href="/own-model">Own your model</a> . <a href="/capture">Capture
checklist</a> . <a href="mailto:hello@khelsutra.guru">hello@khelsutra.guru</a> . <a
href="https://khelsutra.guru">khelsutra.guru</a></div>
705:  function mailtoFallback(p){
717:    return 'mailto:hello@khelsutra.guru?subject=' + encodeURIComponent('KhelSutra alpha

```

```
access') +  
722:   if(f.company && f.company.value) return;           // honeypot: silently drop bots  
749:   window.location.href = mailtoFallback(p); // backend not live yet ? prefilled email
```

17. user (2026-06-22T09:33:26.021Z)

(Bash completed with no output)

18. assistant (2026-06-22T09:33:29.702Z)

The honeypot is `company` with CSS class `hp` ? the doc's \$3 claim says field name is `hp`, which is the class, not the field name. Minor. The `mailtoFallback` confirms progressive enhancement. Let me get the test count properly.

19. user (2026-06-22T09:33:32.589Z)

```
? [VM host] serves the tool screencast as video/mp4 (plays in the post-signup reveal) (2.8056ms)  
? [VM host] serves /own-model (8.7043ms)  
? [VM host] serves the academy flyers (/flyer, /flyer-kn, /flyer-hi) as clean URLs (8.53ms)  
? [VM host] POST /api/request-access persists to SQLite (16.1421ms)  
? [VM host] honeypot + bad email are rejected without persisting (5.5045ms)  
? [VM host] GET /api/request-access -> 405 (1.1964ms)  
? [VM host · N8] a Kannada browser (Accept-Language: kn) gets the kn-rendered page (2.8829ms)  
? [VM host · N8] ?lang= overrides the browser language (2.1961ms)  
? [VM host · N8] the persisted cookie beats Accept-Language (returning user keeps their choice)  
(1.7023ms)  
? [VM host · N8] unsupported/absent language falls back to en (never a locale we can't serve)  
(3.046ms)  
? [VM host · N8] non-localized assets are NOT rewritten (only the 3 marketing pages negotiate)  
(1.4376ms)  
? [VM host] serves the shared catalogs the shim fetches (/locales/*/common.json as JSON)  
(2.7657ms)  
? submit shows an obvious in-flight state (disabled + aria-busy + 'Sending?') (327.5734ms)  
? on failure the button is restored ? never stuck on a frozen 'Sending?' (64.5985ms)  
? after sign-up BOTH videos appear and a ? closes the reveal (68.6015ms)  
? A2: after sign-up the reveal offers a WhatsApp share of the reel (wa.me, safe, localizable)  
(42.9984ms)  
? the auto-close HOLDS while the reel is playing (no mid-watch cut-off) (87.735ms)  
? tests 86  
? suites 0  
? pass 86  
? fail 0  
? cancelled 0  
? skipped 0  
? todo 0  
? duration_ms 2064.9231
```

20. assistant (2026-06-22T09:33:38.090Z)

86 tests pass total across the whole site suite. The doc's specific claim (\$6 P2) is "unit tests (10 green)". Let me check how many tests are specifically in the index.test.js (the Worker unit tests, which is what P2's "10 green" refers to) and the PR #3 context.

21. user (2026-06-22T09:33:39.516Z)

```
? tests 15  
? pass 15  
? fail 0
```

22. user (2026-06-22T09:33:40.601Z)

```
commit f69984786c59f209efdf39c79cabd6c8ff20ce08  
Author: Avi <avi.dullu@gmail.com>  
Date: Sat Jun 20 11:30:00 2026 +0530
```

feat(site): Request-Access Worker + D1 backend (+ 10 unit tests) (#3)

Turns the static site into a Cloudflare Worker with static assets: serves public/ and handles POST /api/request-access -> validate + honeypot-filter + insert into D1 (access_requests). Restructures site/ into public/ (assets), src/ (worker + tests), wrangler.toml, schema.sql.

Tests: `node --test src/` ? 10 green (validation, honeypot, payload sanitise, enum rejection, insert, API/asset routing, 400/405). Pure helpers + a fake DB, so no workerd/D1 needed to run them. Owner runs `wrangler d1 create` + `deploy` (or connects git/Workers Builds) to ship; the form falls back to mailto until then.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

NEXT_STEPS.md		4 +-
docs/REQUEST_ACCESS_FORM.md		6 +--
site/README.md		49 ++++++-----
site/make_og.py		6 ++-
site/package.json		11 +++++
site/{ => public}/capture.html		0
site/{ => public}/index.html		0
site/{ => public}/og.png		Bin
site/schema.sql		18 ++++++
site/src/index.js		88 ++++++
site/src/index.test.js		110 ++++++
site/wrangler.toml		19 ++++++

===PR3 msg===

Turns the static site into a Cloudflare Worker with static assets: serves public/ and handles POST /api/request-access -> validate + honeypot-filter + insert into D1 (access_requests). Restructures site/ into public/ (assets), src/ (worker + tests), wrangler.toml, schema.sql.

Tests: `node --test src/` ? 10 green (validation, honeypot, payload sanitise, enum rejection, insert, API/asset routing, 400/405). Pure helpers + a fake DB, so no workerd/D1 needed to run them. Owner runs `wrangler d1 create` + `deploy` (or connects git/Workers Builds) to ship; the form falls back to mailto until then.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

23. assistant (2026-06-22T09:33:47.480Z)

Confirmed: PR #3 shipped "10 unit tests" (now grown to 15 in index.test.js). The doc's "(10 green)" was accurate at P2 ship time. Also important: PR #3 restructured `site/{ => public}/index.html` ? so the doc's references to `site/index.html` (\$4) were created on 2026-06-20 but became stale on the same day with PR #3, which moved it to `site/public/index.html`. The doc was "Last updated: 2026-06-20" but the path reference wasn't updated.

Now let me verify the PR numbers in the tracker (\$6) and the email-notify "LIVE" claim. The doc references PRs #2 (P0/P1) and the P2 row references a live Worker. Let me check the notify.js and confirm the email notification exists.

24. user (2026-06-22T09:33:48.513Z)

```
1 // Notification adapters (best-effort ? a notify failure must never fail a submission).
2 // Cloudflare path: Email Routing's send_email binding -> a VERIFIED destination (free).
3 // VM path: no-op for now (add SMTP later ? see docs/PORTABILITY.md).
4
5 export function buildSubmissionEmail(r) {
6   const text = [
7     `Segment: ${r.segment || "?"}`,
8     `Name:    ${r.name || ""}`,
9     `Email:   ${r.email || ""}`,
10    `City:    ${r.city || ""}`,
11    r.org_name ? `Academy: ${r.org_name} (${r.courts || "?"} courts)` : null,
```

```

12     `GPU:      ${r.has_gpu || ""}`,
13     `Storage: ${r.storage || ""}`,
14     `Consent: ${r.consent ? "yes" : "no"}`,
15     r.message ? `\\nMessage:\\n${r.message}` : null,
16   ].filter(Boolean).join("\\n");
17   const subject = `New KhelSutra alpha request ? ${r.segment || "?"} . ${r.email ||
"?"}`;
18   return { subject, text };
19 }
20
21 // Cloudflare Email Routing send binding (`[[send_email]]` in wrangler.toml).
22 // `to` must be a verified destination address. Dynamically imports cloudflare:email so
23 // this module stays loadable under Node (the VM path / tests never call into it).
24 export function cfEmailNotify(binding, from, to) {
25   return {
26     async notify(r) {
27       if (!binding || !from || !to) return;
28       const { subject, text } = buildSubmissionEmail(r);
29       const raw =
30         `From: KhelSutra <${from}>\\r\\nTo: ${to}\\r\\nSubject: ${subject}\\r\\n` +
31         `MIME-Version: 1.0\\r\\nContent-Type: text/plain; charset=utf-8\\r\\n\\n${text}`;
32       const { EmailMessage } = await import("cloudflare:email");
33       await binding.send(new EmailMessage(from, to, raw));
34     },
35   };
36 }
37
38 export function noopNotify() {
39   return { async notify() {} };
40 }
41

```

25. user (2026-06-22T09:33:50.261Z)

commit 2274faa409b62981e6afba5fffd02101d4d3ab338
 Author: Avi <avi.dullu@gmail.com>
 Date: Sat Jun 20 11:23:06 2026 +0530

feat(site): segmented Request-Access form (academy/individual + GPU + storage discovery) (#2)

Replaces the mailto CTA with a real form: segments academy vs individual, asks the compute (GPU?) and storage (where do your videos live?) questions that map onto the BYO-compute / BYO-storage architecture forks, plus a free-text message and consent. Posts JSON to /api/request-access (Worker + D1 ? next PR) and falls back to a prefilled mailto until that ships, so it works from the first re-upload. Honeypot anti-spam; academy-only fields (club name, #courts) reveal on segment select.

Brings the engine repo's doc discipline to khelsutra: docs/REQUEST_ACCESS_FORM.md (tracked design doc ? decisions-locked + progress tracker + DoD) and NEXT_STEPS.md.

Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

```

NEXT_STEPS.md          | 34 ++++++++
docs/REQUEST_ACCESS_FORM.md | 106 +++++
===P0/P1 PR2 subject===
feat(site): segmented Request-Access form (academy/individual + GPU + storage discovery) (#2)

```

26. assistant (2026-06-22T09:33:56.813Z)

PR #2 (P0+P1) and PR #3 (P2 Worker + tests) confirmed. Email-notify is implemented (notify.js + wrangler.toml `[[send\_email]]`) ? so the P3 "email-notify LIVE" claim is code-backed. The "LIVE on Cloudflare" deployment claims are owner-run runtime facts I can't verify from the repo (unverifiable).

Now let me check the P3/P4 status and Turnstile. The doc says Turnstile still pending (P3 ?) and admin export pending (P4 ?). Let me verify Turnstile and admin view are indeed absent, and check the `server.js` VM host mentioned in tests (the doc doesn't mention it but it's relevant to whether the doc is superseded).

27. user (2026-06-22T09:34:05.860Z)

```
===Turnstile?===
docs/REQUEST_ACCESS_FORM.md:66:- **Anti-spam:** honeypot first; add **Cloudflare Turnstile** in
P2 if spam appears.
docs/REQUEST_ACCESS_FORM.md:85:| P3 | Anti-spam (Turnstile) + per-submission email notify | No |
? | **email-notify LIVE** (per-submission to `hello@`); Turnstile still ? |
docs/REQUEST_ACCESS_FORM.md:97:1. Turnstile now or wait for spam? (Default: honeypot now,
Turnstile if needed.)
===admin/export view?===
===server.js routes===
2:// Serves public/ and POST /api/request-access into SQLite (node:sqlite), using the
96:async function serveStatic(pathname, req, searchParams, res) {
97:  const rel = pathname === "/" ? "index.html"
98:    : pathname === "/capture" ? "capture.html"
99:    : pathname === "/own-model" ? "own-model.html"
100:   : pathname === "/flyer" ? "flyer.html"
101:   : pathname === "/flyer-kn" ? "flyer-kn.html"
102:   : pathname === "/flyer-hi" ? "flyer-hi.html"
103:   : pathname.replace(/^\/+/, "");
127:  const pathname = decodeURIComponent(parsed.pathname);
128:  if (pathname === "/api/request-access") {
132:    return await serveStatic(pathname, req, parsed.searchParams, res);
```

28. assistant (2026-06-22T09:34:14.213Z)

Confirmed:

- Turnstile: NOT implemented (only referenced in the doc). P3 ? status correct. Note: S4 says "add Cloudflare Turnstile in P2" but the tracker S6 lists Turnstile under P3 ? minor internal inconsistency in the doc, but the P3 status itself is accurate.
- Admin/export view (P4): no admin/export code ? P4 ? correct.
- The email-notify is wired in code ? P3 "email-notify LIVE" is code-backed (deployment is owner-runtime, unverifiable but consistent).

Now let me verify the doc's "P5" claim ? the doc only lists P0?P4, fine. Let me check the email notify destination: the doc P3 says "per-submission to `hello@" but wrangler.toml shows `NOTIFY_TO = "avi.dullu@gmail.com"`. Minor factual drift. Let me also verify the S4 D1 schema column list claim against actual schema (already read ? matches). And check the consent label wording and the doc's claim that the form replaces the mailto CTA (DoD).

Let me verify the "Request access" CTA in the header/hero now scrolls to `#join` (doc S4) and the consent checkbox text.

29. user (2026-06-22T09:34:16.746Z)

```
===#join + Request access CTA===
269:      <a class="btn btn-dark" href="#join" data-i18n="site.nav.requestAccess">Request
access</a>
282:      <a class="btn btn-primary" href="#join" data-i18n="site.hero.ctaPrimary">Request
alpha access</a>
284:      <a class="btn btn-ghost" href="#join" data-i18n="site.hero.ctaReel">See it in action
+ a real reel</a>
519:      <details><summary data-i18n="site.faq.q7.q">How do I join?</summary><p
data-i18n-html="site.faq.q7.a">It's an invite-only private trial. Just <a href="#join">request
access</a> using the form ? it takes a minute, you'll see a real sample reel right away, and
we'll be in touch about recording your own court.</p></details>
524:<section id="join" style="padding-bottom:90px">
527:      <div class="kicker">Request access</div>
604:      <button class="btn btn-primary" type="submit" style="width:100%;justify-
```

```

content:center">Request access</button>
===consent===
7:<meta name="description" content="KhelSutra turns a full badminton match into a clean index of
rallies with the dead time stripped out ? a highlight reel you can watch and download. Invite-
only alpha, Bengaluru." />
9:<meta property="og:description" content="A full match becomes a clean highlight reel ? only
the real play, no dead time. Invite-only alpha for academies and players in Bengaluru." />
17:<meta name="twitter:description" content="A full match becomes a clean highlight reel ? only
the real play, no dead time. Invite-only alpha, Bengaluru." />
203: .af .consent{display:flex;align-items:flex-start;gap:.5rem}
204: .af .consent input{margin-top:.2rem;accent-color:var(--green)}
278:     <span class="eyebrow" data-i18n="site.hero.eyebrow">Invite-only alpha .
Bengaluru</span>
282:     <a class="btn btn-primary" href="#join" data-i18n="site.hero.ctaPrimary">Request
alpha access</a>
450:     <div class="kicker" data-i18n="site.ask.kicker">Coming next to the alpha</div>
452:     <p data-i18n="site.ask.intro">Your match is already a clean index of rallies ? so the
natural next step is to let you build any reel just by describing it, in plain English. This is
rolling out to alpha users.</p>
514:     <details><summary data-i18n="site.faq.q2.q">Can I choose which rallies go in the
reel?</summary><p data-i18n="site.faq.q2.a">Today your reel includes the rallies KhelSutra
found, with the dead time removed. Choosing by length, count, order or which game ? just by
asking in plain English ? is rolling out to alpha users next; shot-type and player queries
follow as the deeper analysis arrives.</p></details>
===hello@ in notify path===
site/wrangler.toml:28:NOTIFY_TO = "avi.dullu@gmail.com"

```

30. assistant (2026-06-22T09:34:21.470Z)

Confirmed:

- §4 "header + hero 'Request access' buttons scroll to the `#join` form; `#join` becomes the form" ? CONFIRMED (lines 269, 282, 284 all `href="#join"`; line 524 `

- The doc P3 says email-notify goes "to `hello@" but the actual notify destination is `avi.dullu@gmail.com` (wrangler.toml:28). Minor factual drift, low/medium severity.

Let me verify the consent checkbox text and the GPU/storage field labels exist as the §3 spec describes, and confirm the GPU options. Let me read the form section directly.

31. user (2026-06-22T09:34:22.690Z)

```

540         </div>
541
542         <div class="row">
543             <label for="af-name">Name</label>
544             <input id="af-name" name="name" type="text" autocomplete="name" required>
545         </div>
546         <div class="row">
547             <label for="af-email">Email <span class="hint">(so we can reach
you)</span></label>
548             <input id="af-email" name="email" type="email" autocomplete="email" required>
549         </div>
550         <div class="row">
551             <label for="af-city">City <span class="hint">(optional)</span></label>
552             <input id="af-city" name="city" type="text">
553         </div>
554
555         <div class="row acad">
556             <label for="af-org">Academy / club name <span
class="hint">(optional)</span></label>
557             <input id="af-org" name="org_name" type="text">
558         </div>
559         <div class="row acad">
560             <label for="af-courts">How many courts do you record? <span
class="hint">(optional)</span></label>
561             <select id="af-courts" name="courts">

```

```

562         <option value="">Select?</option>
563         <option value="1">1 court</option>
564         <option value="2-4">2?4 courts</option>
565         <option value="5+">5+ courts</option>
566     </select>
567 </div>
568
569     <div class="row">
570         <label data-i18n="site.join.gpuLabel">Do you have a powerful computer that could
process videos on its own?</label>
571         <div class="hint" style="margin:-.2rem 0 .5rem" data-i18n="site.join.gpuHint">A
gaming PC or laptop with a graphics card. No worries if not ? we can run it in the cloud for
you.</div>
572         <div class="opts">
573             <label><input type="radio" name="has_gpu" value="yes"><span
data-i18n="site.join.gpuYes">Yes ? a gaming PC or laptop with a graphics card</span></label>
574             <label><input type="radio" name="has_gpu" value="unsure"><span
data-i18n="site.join.gpuUnsure">Not sure</span></label>
575             <label><input type="radio" name="has_gpu" value="no"><span
data-i18n="site.join.gpuNo">No</span></label>
576             <label><input type="radio" name="has_gpu" value="prefer-cloud"><span
data-i18n="site.join.gpuCloud">No ? please run it in the cloud for me</span></label>
577         </div>
578     </div>
579
580     <div class="row">
581         <label><span data-i18n="site.join.storageLabel">Where do your match videos live?
How would you share them?</span> <span class="hint" data-i18n="site.join.storageHint">(pick
any)</span></label>
582         <div class="opts">
583             <label><input type="checkbox" name="storage" value="gdrive"><span
data-i18n="site.join.storageGdrive">Google Drive</span></label>
584             <label><input type="checkbox" name="storage" value="onedrive"><span
data-i18n="site.join.storageOnedrive">OneDrive / Microsoft 365</span></label>
585             <label><input type="checkbox" name="storage" value="dropbox"><span
data-i18n="site.join.storageDropbox">Dropbox</span></label>
586             <label><input type="checkbox" name="storage" value="bucket"><span
data-i18n="site.join.storageBucket">A cloud storage bucket ? advanced (e.g. Amazon S3, Google
Cloud)</span></label>
587             <label><input type="checkbox" name="storage" value="local"><span
data-i18n="site.join.storageLocal">SD card or the device itself</span></label>
588             <label><input type="checkbox" name="storage" value="unsure"><span
data-i18n="site.join.storageUnsure">Not sure</span></label>
589         </div>
590     </div>
591
592     <div class="row">
593         <label for="af-msg">Anything else? <span class="hint">(how you record, what
you'd want?)</span></label>
594         <textarea id="af-msg" name="message"></textarea>
595     </div>
596
597     <div class="row consent">
598         <input id="af-consent" name="consent" type="checkbox" value="yes">
599         <label for="af-consent" style="margin:0;font-weight:400;color:var(--mut)">Okay
to email me about the KhelSutra alpha.</label>
600     </div>
601
602     <input class="hp" type="text" name="company" tabindex="-1" autocomplete="off"
aria-hidden="true">
603
604     <button class="btn btn-primary" type="submit" style="width:100%;justify-
content:center">Request access</button>

```

The form spec in §3 matches the live form: segment radio, name/email/city, academy org+courts (`1`/`2-4`/`5+`), GPU radio (`yes`/`unsure`/`no`/`prefer-cloud`), storage checkboxes (`gdrive`/`onedrive`/`dropbox`/`bucket`/`local`/`unsure`), message textarea, consent checkbox, honeypot. All field values match `index.js` enums.

The one §3 discrepancy: doc lists the honeypot field as `hp` ? but `hp` is the CSS **class** (line 602), and the actual field **name** is `company`. The doc's §3 "Field" column for the honeypot row says "`hp`" which is wrong as a field name. Low severity (the doc is describing the form, and `hp` appears as the class).

Now let me check whether referenced sibling docs exist (PORTABILITY.md is referenced in code; the doc itself references engine-repo docs which are out-of-repo, so those are unverifiable cross-repo). Let me verify NEXT_STEPS.md exists and check the doc's overall lifecycle/archive question ? whether P1-P3 are effectively done (the DoD says "archived when P1-P3 land").

33. user (2026-06-22T09:34:35.829Z)

```
NEXT_STEPS.md
docs/PORTABILITY.md
===archives===
FRIENDLY_AND_MULTILINGUAL.md
===NEXT_STEPS request-access mention===
6:checklist + OG image + a segmented Request-Access form (academy/individual + GPU + storage
+ free
14:`docs/REQUEST_ACCESS_FORM.md`, `docs/PORTABILITY.md`.
```

34. assistant (2026-06-22T09:34:42.976Z)

All referenced in-repo docs exist (NEXT_STEPS.md, docs/PORTABILITY.md). The doc's status header is `IN PROGRESS`, which is accurate ? P1/P2 done (?), P3 partial (?? email done, Turnstile pending), P4 not started (?). So it is genuinely still an in-progress tracker, NOT yet an archive candidate (unlike the FRIENDLY_AND_MULTILINGUAL one which was complete). The DoD says "archived when P1?P3 land" ? P3 hasn't fully landed (Turnstile ?), so it correctly remains live.

Let me do a final check on the "Last updated: 2026-06-20" claim ? the form has since been substantively reworked (PR #75 "A6 ? plain-language GPU/storage questions", localization added). The doc §3 still shows the old terse labels ("GPU machine for local processing?") whereas the live form uses plain-language i18n labels. This is a minor staleness. Let me confirm PR #75 changed these labels.

35. user (2026-06-22T09:34:44.720Z)

```
commit aa8d5e979adccddff7646b2d5e7ba7d7cd9e2aac
Author: Avi <avi.dullu@gmail.com>
Date: Mon Jun 22 00:34:21 2026 +0530
```

feat(site): A6 ? plain-language Request-Access GPU/storage questions (#75)

De-jargon the intimidating access-form copy so a non-technical coach/parent isn't scared off:

- "Do you have a GPU machine that could process video locally?" ? "Do you have a powerful computer that could process videos on its own?" + a reassurance hint ("A gaming PC or laptop with a graphics card. No worries if not ? we can run it in the cloud for you.").
- Storage: drop the bare "(S3/GCS)" ? "A cloud storage bucket ? advanced (e.g. Amazon S3, Google Cloud)"; "SD card / local drive" ? "SD card or the device itself"; etc.
- Wired via the shim (site.join.*, data-i18n on wrappers that keep the radio/checkbox <input>s intact), so it's translation-ready (en for now; form kn/hi is a later increment).
- Backend enum contract unchanged: field names + every has_gpu/storage value preserved

```
===date of PR75===
2026-06-22 00:34:21 +0530
```

36. assistant (2026-06-22T09:34:59.088Z)

PR #75 (2026-06-22) reworded the form copy but kept the backend enum contract identical. So §3's *field values* remain correct; only the human-facing label text in §3 ("GPU machine for local processing?") is stale prose vs the live plain-language copy. The §3 table is primarily a field/enum spec, so this is low severity.

Also: the doc's §8 Open Question 3 ("Do we localize the form (Kannada/Hindi)... Default: English now") ? the surrounding site has since been fully localized to 6 languages, and the form labels are now wired through i18n (`data-i18n`), though form translations themselves are noted as "a later increment." This is a minor stale framing, not wrong.

I have enough to compile the audit. Let me summarize the findings.

Verified facts:

- Route `POST /api/request-access` ? correct (index.js:80).
- D1 table `access\_requests` with the exact column list in §4 ? correct (schema.sql).
- Worker `hidden-king-khelsutra` + D1 `khelsutra-access` ? correct in wrangler.toml.
- Form spec §3 (segments, GPU/storage/courts enum values, consent, honeypot) ? matches index.js + index.html.
- §4 "#join becomes the form; header+hero buttons scroll to #join" ? correct.
- PRs #2 (P0/P1), #3 (P2 Worker + 10 tests) ? correct.
- P3 ? (email-notify built in notify.js + wrangler `[[send\_email]]`; Turnstile absent) ? correct.
- P4 ? (no admin/export code) ? correct.

Stale/wrong:

- §4 references `site/index.html` (frontend) ? file is now `site/public/index.html`; `site/index.html` does NOT exist (moved in PR #3 on 2026-06-20, same day as "Last updated"). Medium ? a reader would look in the wrong place.
- §3 honeypot field listed as `hp` ? actual field **name** is `company` (`hp` is the CSS class). Low/medium ? wrong if read as field name.
- P3 row says email goes to `hello@` ? actual `NOTIFY\_TO` is `avi.dullu@gmail.com`. Low.
- §3/§8 form label prose is pre-PR#75 jargon; live copy is plain-language + i18n-wired. Low.
- §4 vs §6 internal inconsistency: §4 says "add Turnstile in P2", tracker puts it in P3. Low.

Unverifiable (owner-runtime, not in repo): "LIVE on Cloudflare", "persists + emails per submission", deployed live on khelsutra.guru.

37. user (2026-06-22T09:35:31.680Z)

Structured output provided successfully

38. assistant (2026-06-22T09:35:44.791Z)

Audit complete. The doc `docs/REQUEST\_ACCESS\_FORM.md` is in good shape overall (minor-staleness) ? it's a still-live IN-PROGRESS tracker whose core claims (the `/api/request-access` route, D1 schema, form spec/enums, PR numbers, and P1-done/P2-done/P3-partial/P4-todo statuses) all verify against the code, so it is correctly NOT an archive candidate (unlike the already-archived multilingual tracker).

The real fix-worthy items:

- Medium: §4 cites the frontend as `site/index.html`, but the file is now `site/public/index.html` (moved by PR #3 on the very day the doc was last updated; `site/index.html` no longer exists).
- Low: §3 lists the honeypot field name as `hp`, but `hp` is the CSS class ? the field is named `company` (what `isBot` actually checks).
- Low: P3 says email-notify goes to `hello@`, but `NOTIFY\_TO` is `avi.dullu@gmail.com`.
- Low: §4 says Turnstile lands "in P2" while the tracker puts it in P3; and §3 label prose predates PR #75's plain-language rewrite (enum values unchanged, so harmless).

Deployment claims ("LIVE on Cloudflare", persists+emails) are owner-runtime and unverifiable from the checkout ? flagged as such rather than assumed correct.